



# VISITOR DESIGN PATTERN

24A02 - Group 3 - 1<sup>st</sup> pattern



# VISITOR DESIGN PATTERN

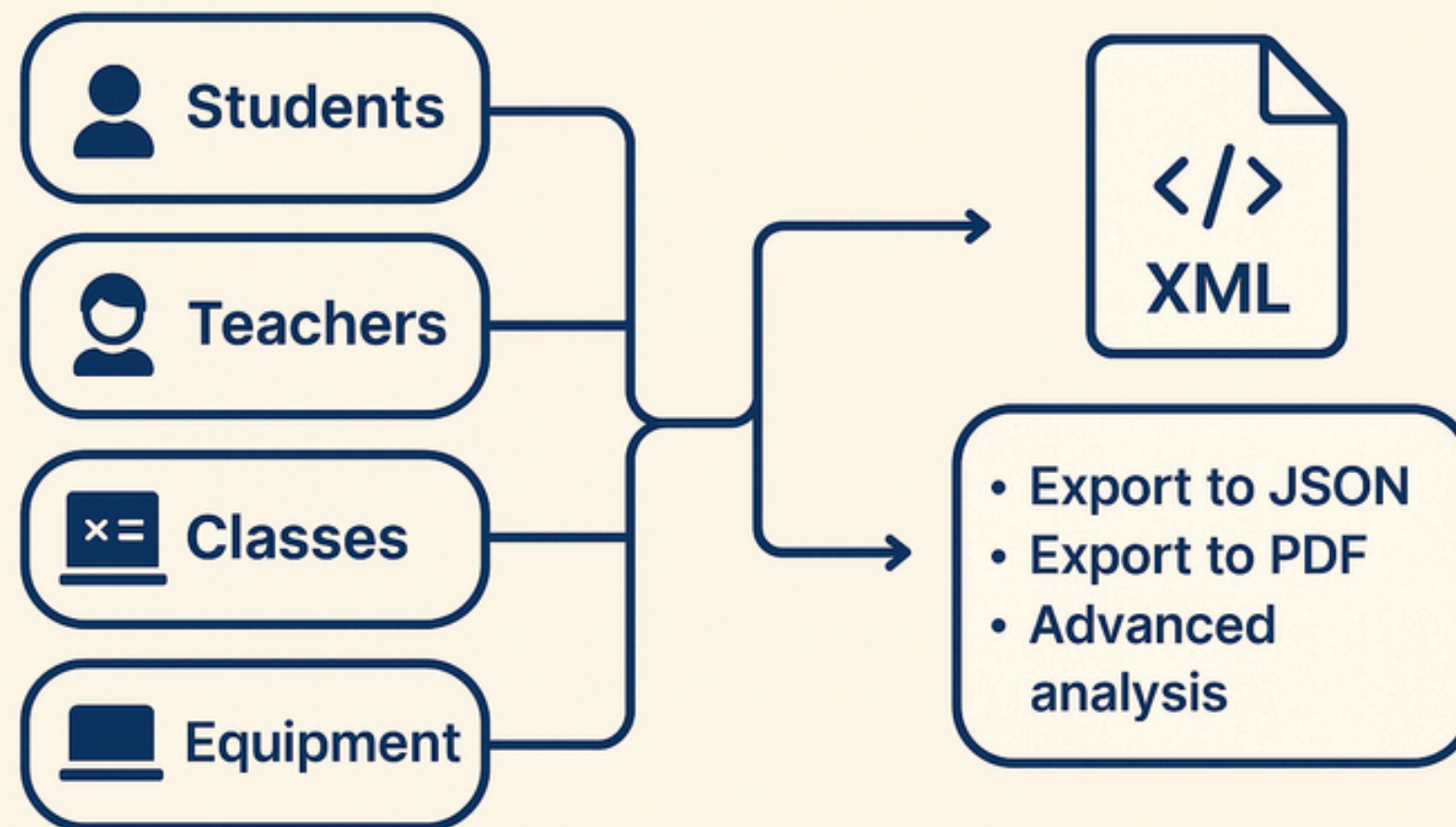


1. **Motivation – An initial problem**
2. **The naïve solution**
3. **A new approach: Visitor**
4. **Structure of Visitor**
5. **Implementation**
6. **Pros and Cons**
7. **Real-world Application**
8. **Quiz**



# Motivation: School Management App

## Exporting School Management Application Data as Graph to XML



Let's assume you or your group is developing a school management application with various components such as students, teachers, classes, and equipment. The main function of the app is to:

- **Export the data** of this graph to XML format.
- In the future, potentially add **export to JSON, PDF, or advanced analysis.**

# Naïve Solution

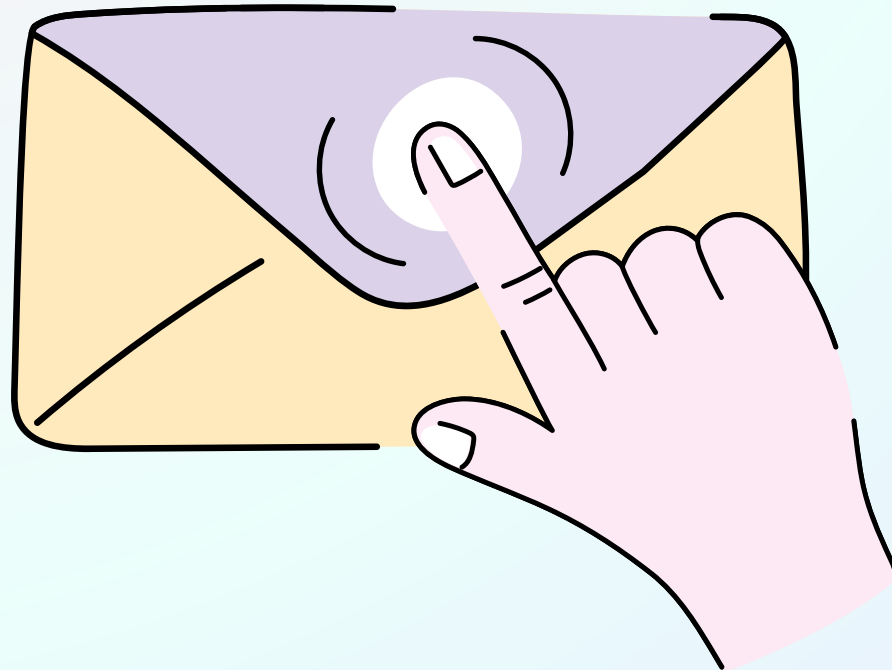
Add the methods: **exportToXML()**, **exportToJSON()** and **exportToPDF()** methods to each node class (such as `Classes`, `Teacher`, `Student`) and use recursion to traverse the graph, calling these methods on each node



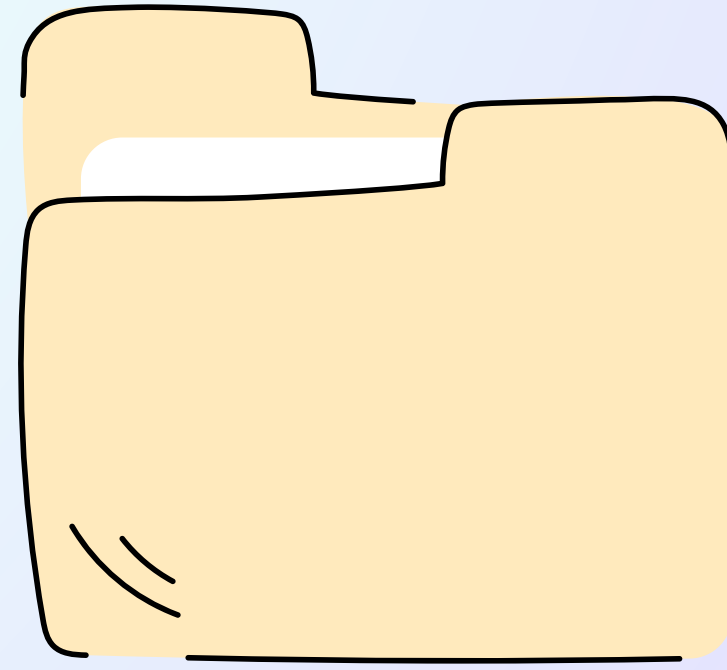
```
class Student{  
public:  
    std::string toXML();  
    std::string toPDF();  
    std::string toJSON();  
}
```



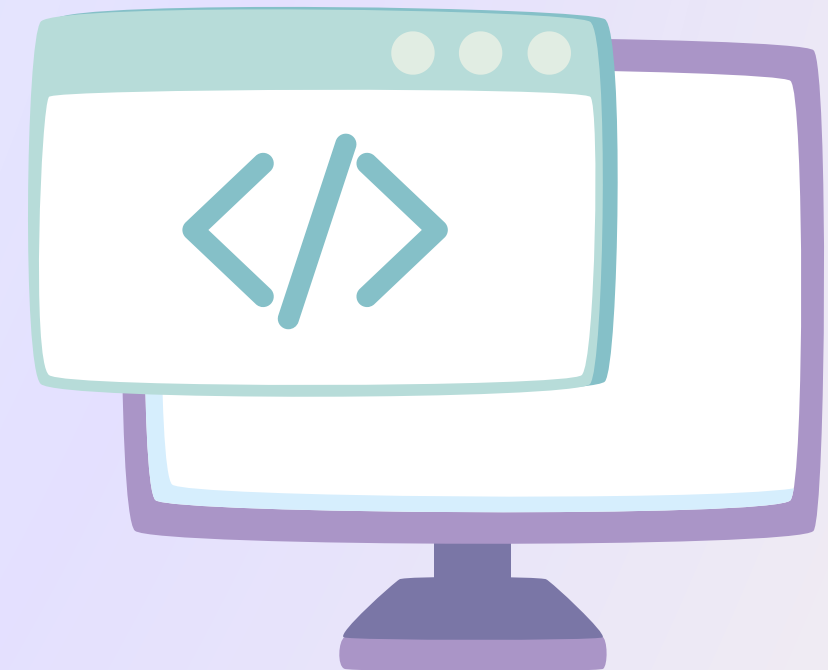
# Issues with this Solution



**Violation of the  
Open/Closed Principle**

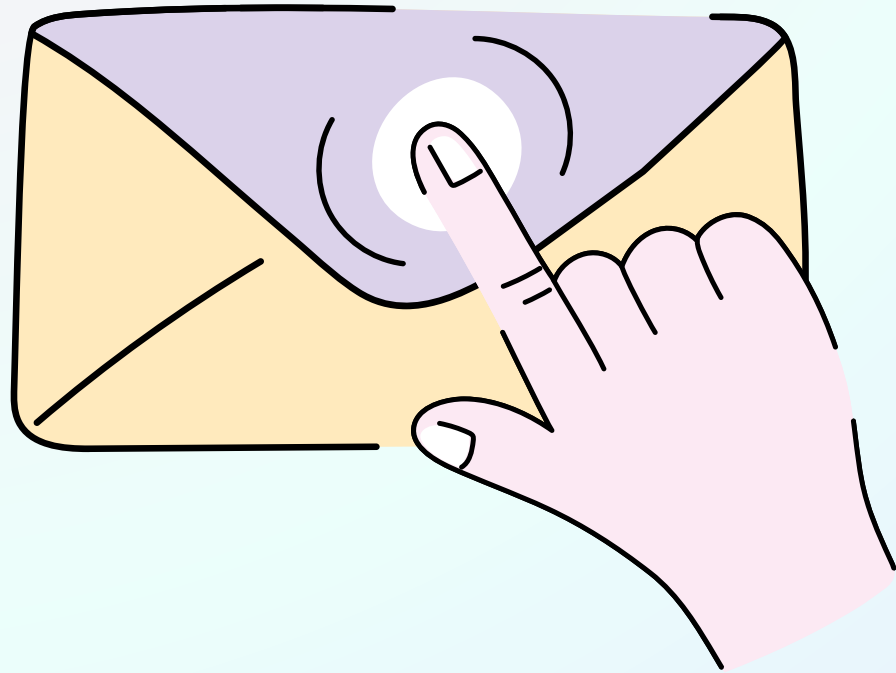


**Risk of Breaking  
Production Code**



**Inappropriate  
Responsibility**

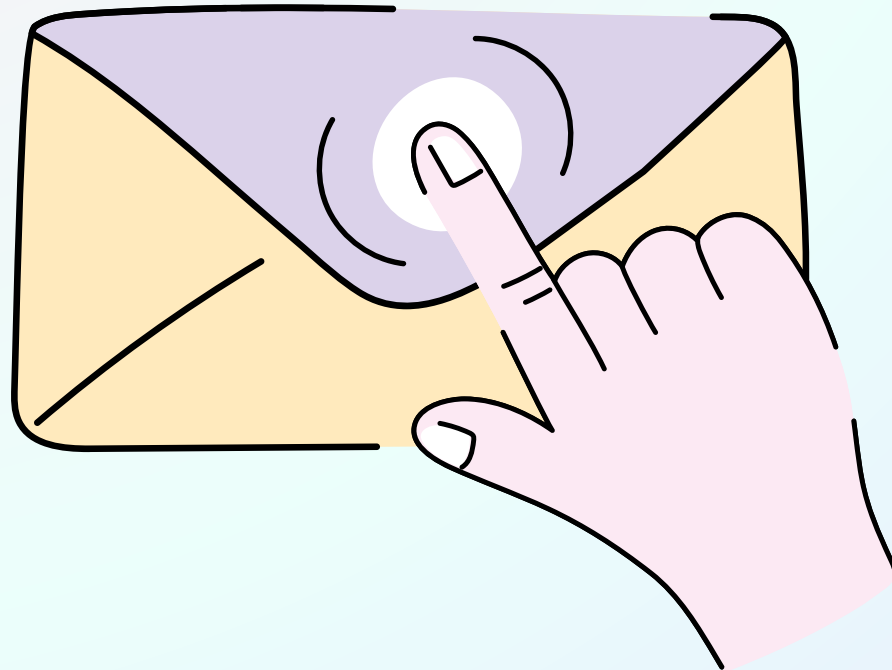
# Issues with this Solution



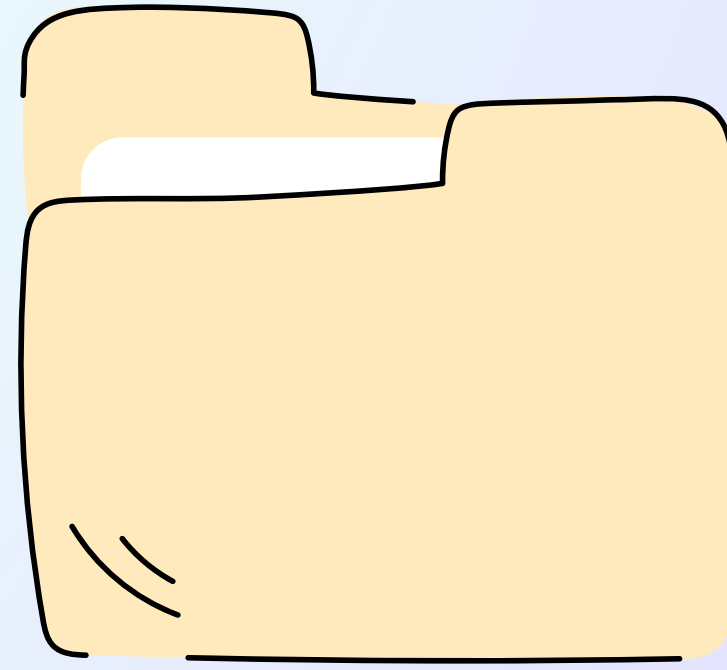
Each new requirement forces code changes, increasing the risk of errors in the product system.

**Violation of the  
Open/Closed Principle**

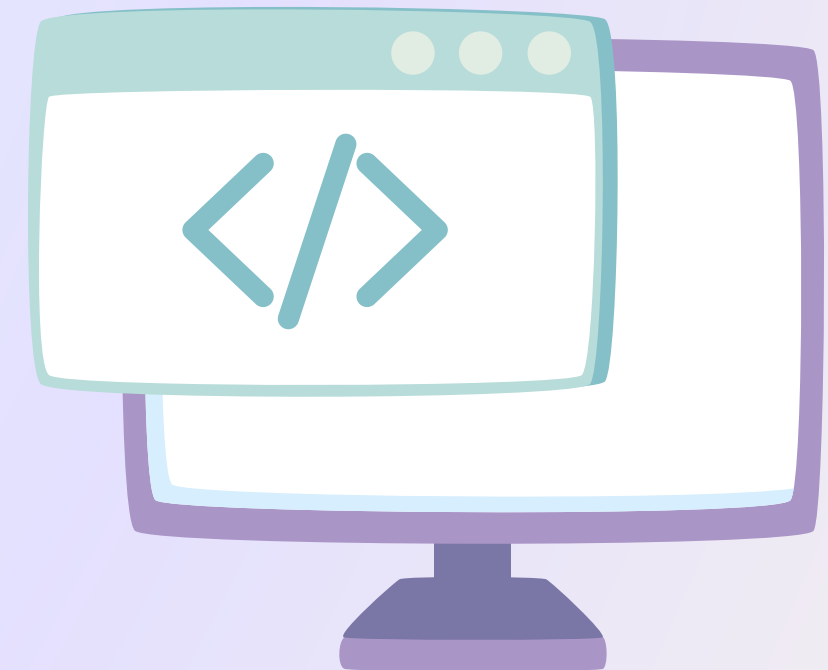
# Issues with this Solution



**Violation of the  
Open/Closed Principle**

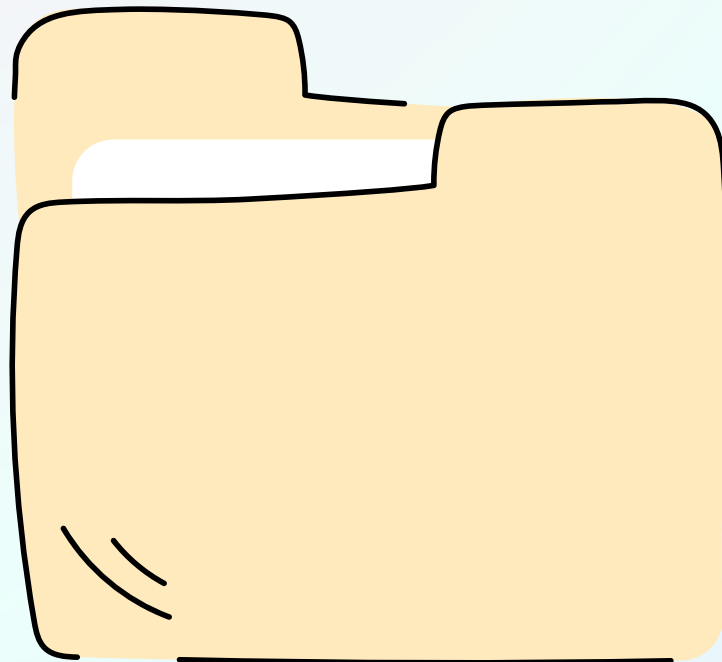


**Risk of Breaking  
Production Code**



**Inappropriate  
Responsibility**

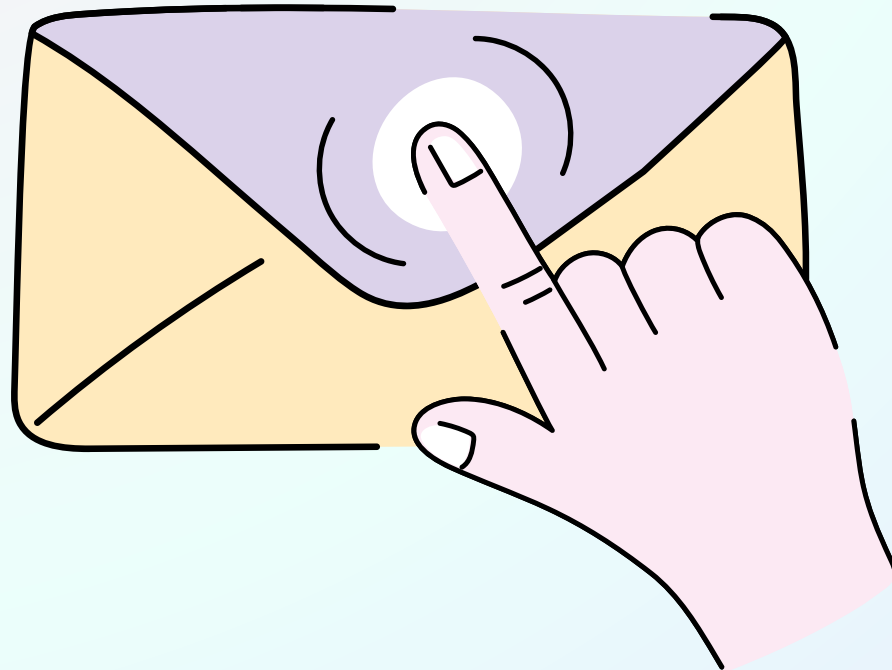
# Issues with this Solution



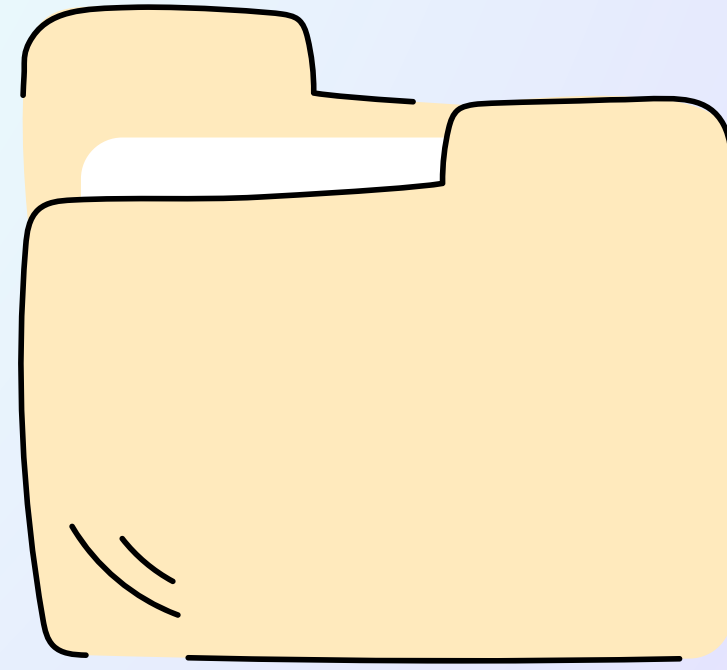
**Risk of Breaking  
Production Code**

The node classes are already in production and modifications could introduce potential bugs.

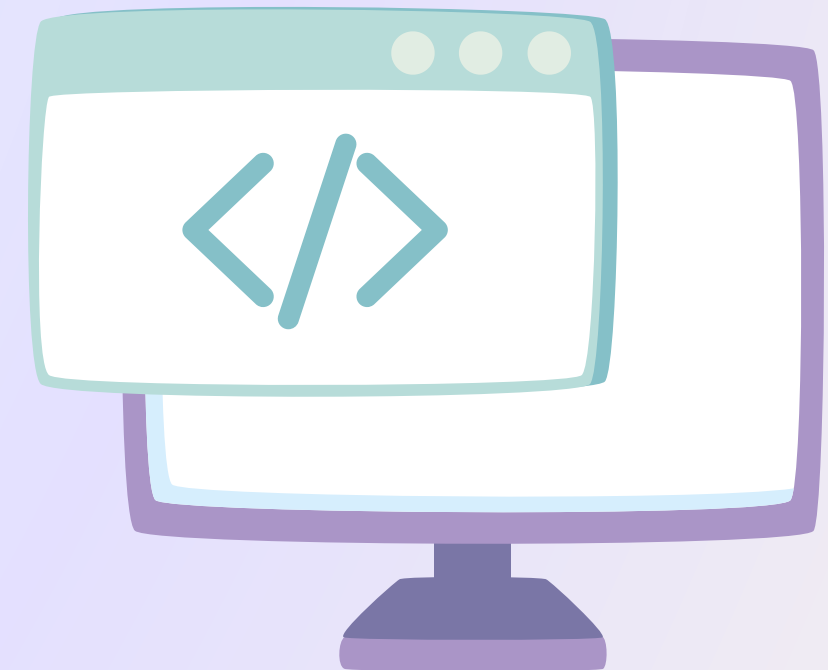
# Issues with this Solution



**Violation of the  
Open/Closed Principle**

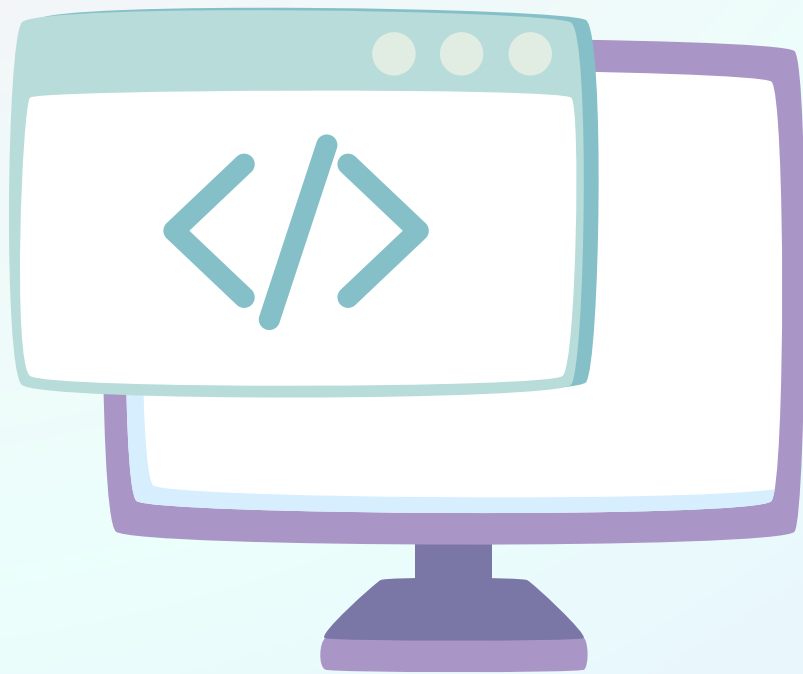


**Risk of Breaking  
Production Code**



**Inappropriate  
Responsibility**

# Issues with this Solution



**Inappropriate  
Responsibility**

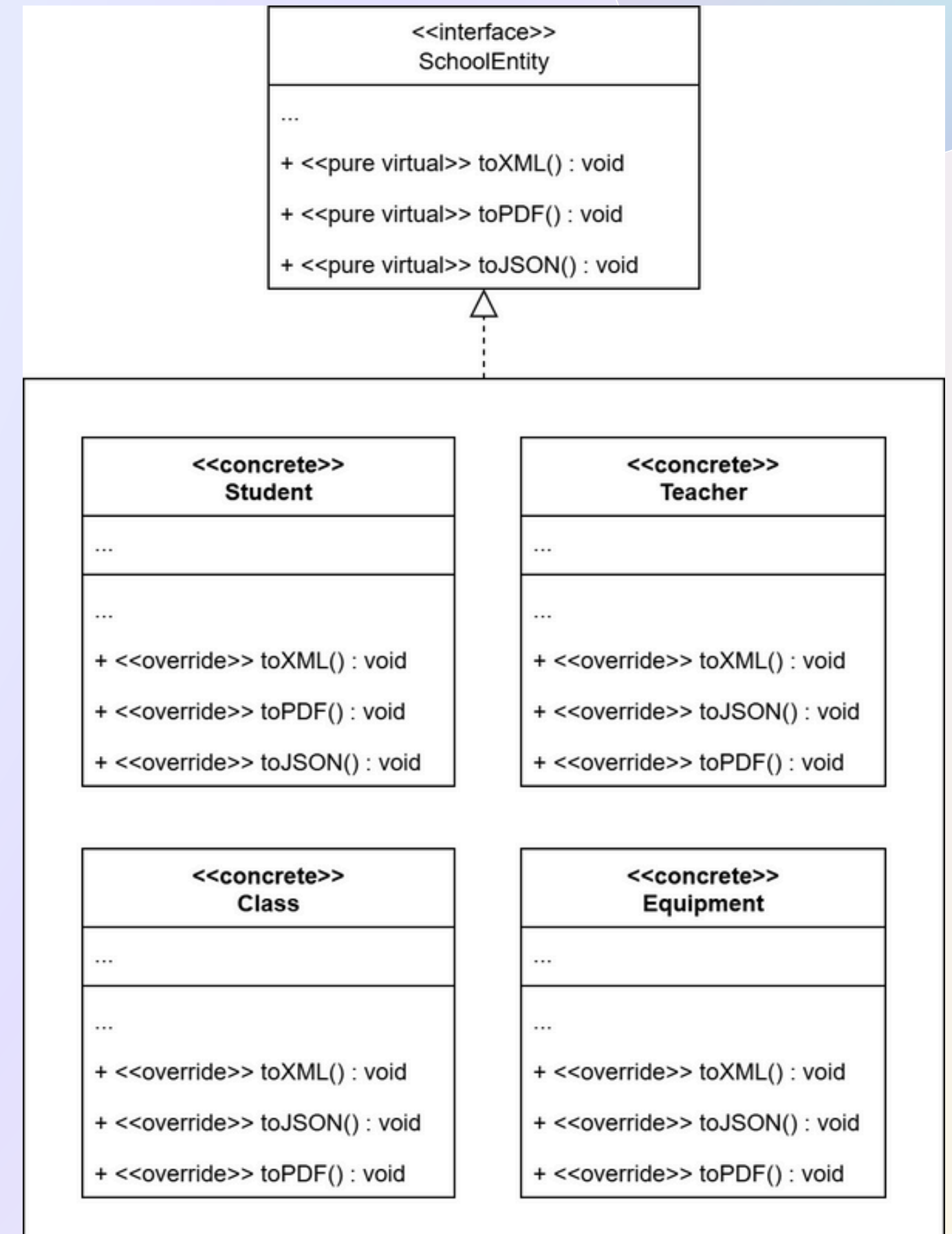
The code becomes harder to **maintain** and prone to logical errors due to a violation of the **Single Responsibility Principle** (each class should have only one responsibility; having multiple responsibilities makes maintenance difficult).

# Diagram of Naïve Solution

**Violation of the  
Open/Closed Principle**

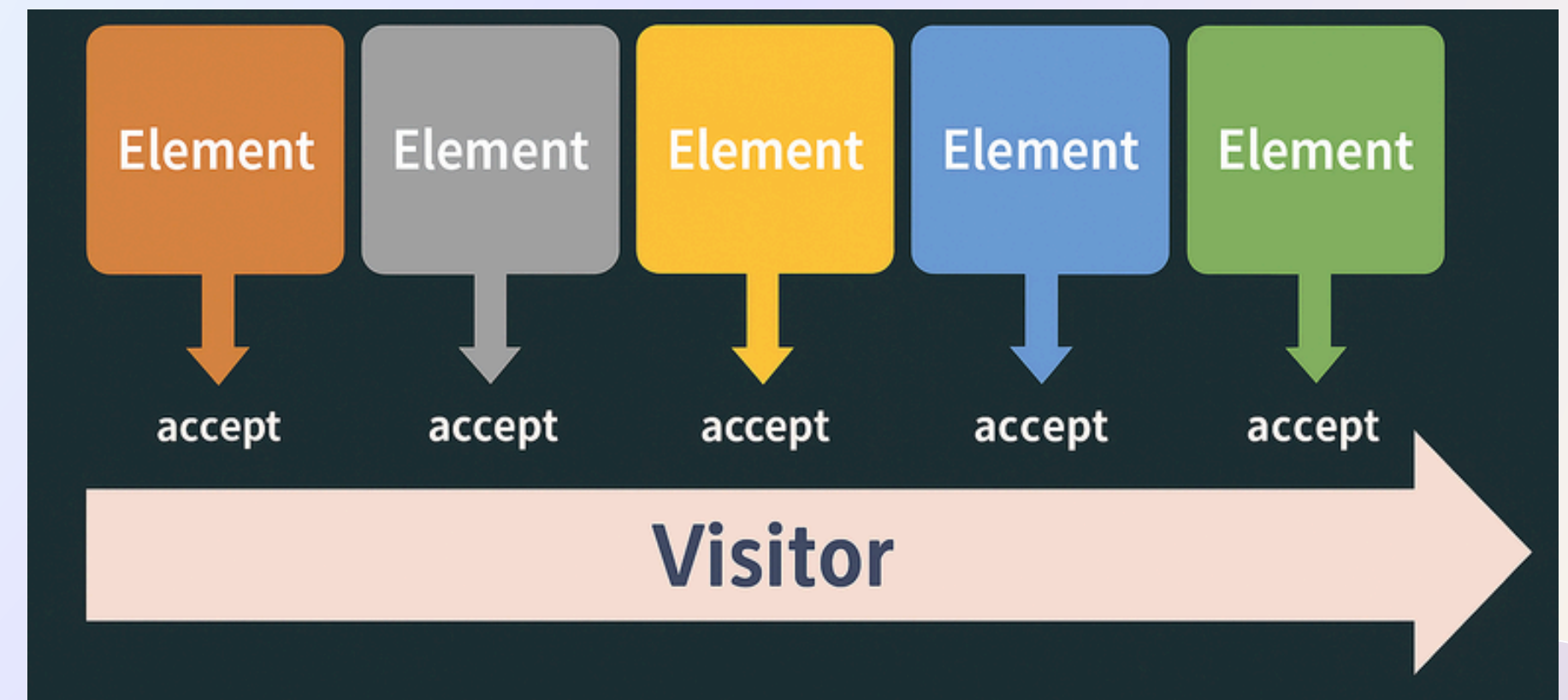
**Risk of Breaking  
Production Code**

**Inappropriate  
Responsibility**



# New Approach: Visitor

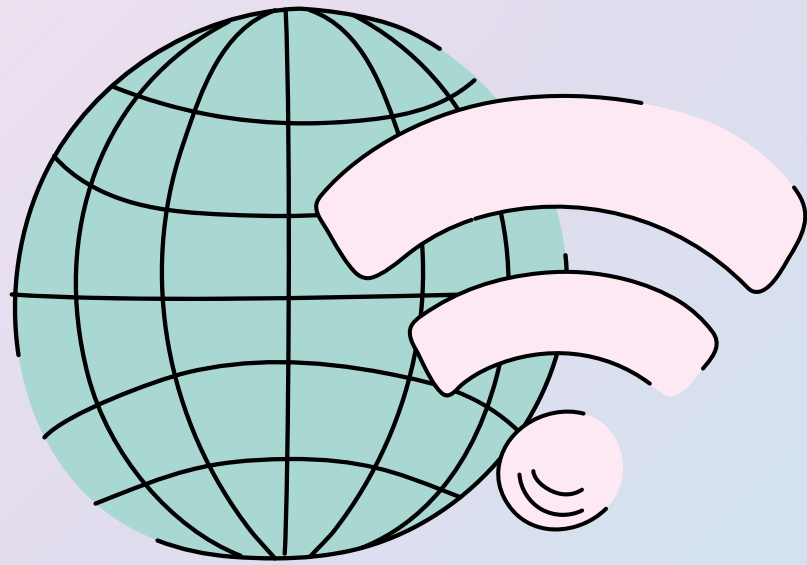
In object-oriented design, the Visitor pattern is a design pattern that allows defining operations on a **set of heterogeneous** (in terms of type) **objects** without modifying the definitions of their classes.



# New Approach: Visitor

To achieve this, the Visitor pattern defines operations in separate classes called **visitor classes**, which enable the separation of operations from the objects they act upon. For each new operation added, a corresponding visitor class is created.





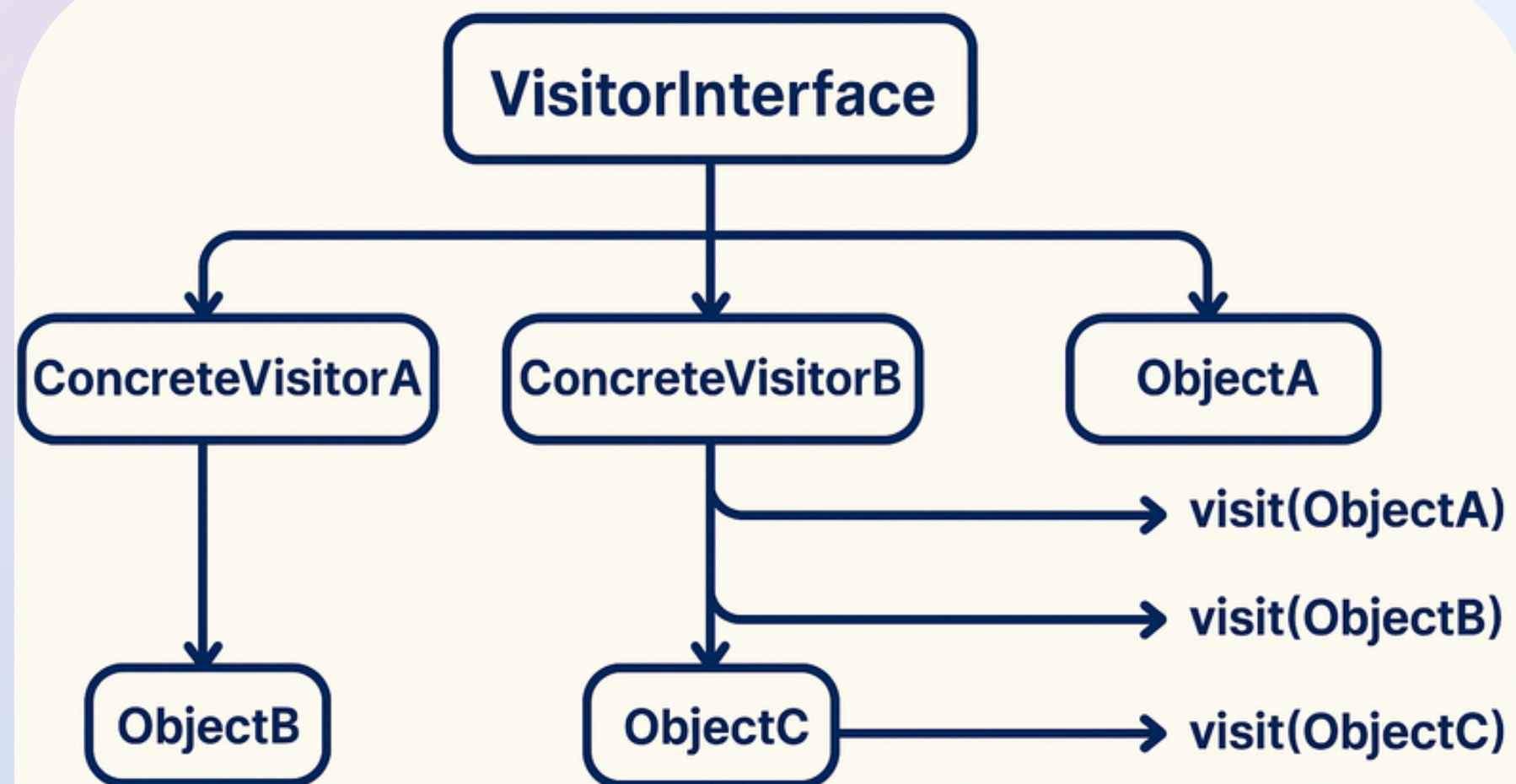
# Structure of the VISITOR Design Pattern

***Visitor Classes***

***Classes of objects  
in the object collection***

***The Client***

# Visitor Classes



- Visitor classes handle operations that work with objects. They all follow a shared interface named “**VisitorInterface**” which includes methods like “**visit( ObjectType object )**”
- For each object type, there's a corresponding “**visit**” method to process that specific type.

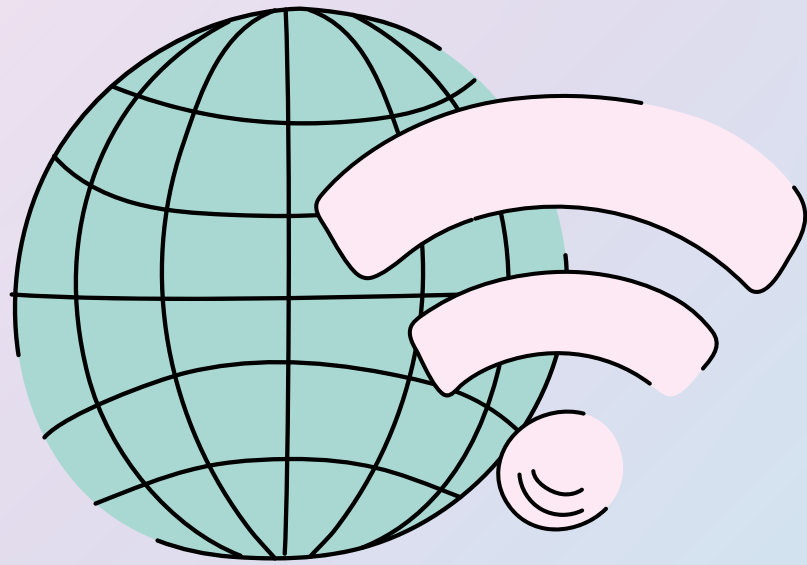
# Visitor Classes

## ●●● ExampleVisitorClass.h

```
class Visitor {  
public:  
    virtual void visitClass(Class* c) = 0;  
    virtual void visitStudent(Student* s) = 0;  
    virtual void visitTeacher(Teacher* t) = 0;  
    virtual void visitEquipment(Equipment* e) = 0;  
    virtual ~Visitor() = default;  
};
```

## ●●● ExampleVisitorClass.h

```
class XMLExportVisitor : public Visitor {  
public:  
    void visitClass(Class* c) override;  
    void visitStudent(Student* s) override;  
    void visitTeacher(Teacher* t) override;  
    void visitEquipment(Equipment* e) override;  
}
```



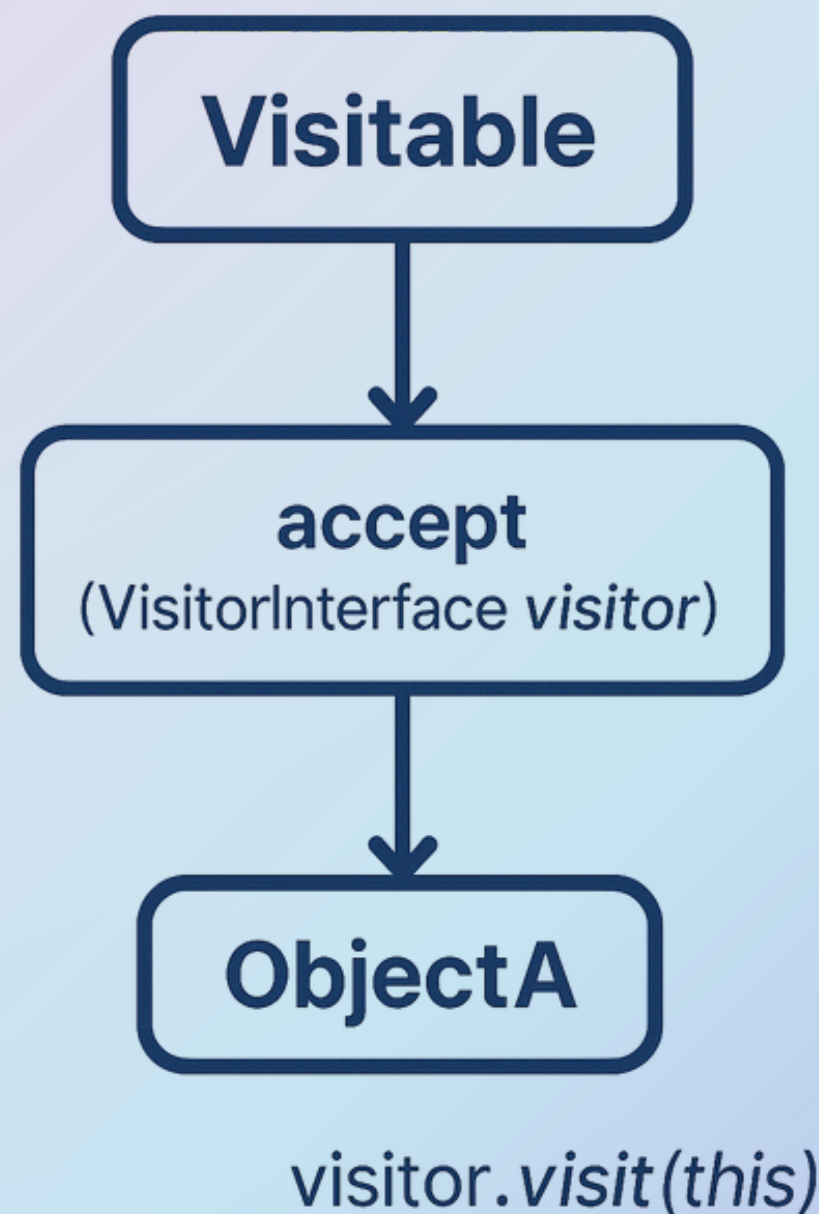
# Structure of the VISITOR Design Pattern

***Visitor Classes***

***Classes of objects  
in the object collection***

***The Client***

## Classes of objects in the object collection

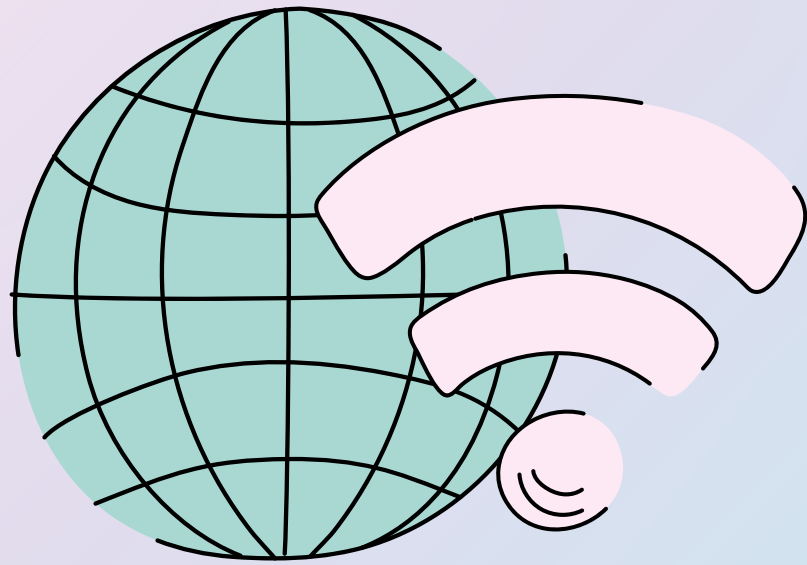


- For the operations defined in the visitor classes to be executed, the classes of the objects in the collection must implement the “**Visitable**” interface, which includes a method: “**accept(VisitorInterface visitor)**”
- In the implementation of this method, the visitor's “**visit**” method is called with the calling object itself passed as the parameter.

# ***Classes of objects in the object collection***

## ●●● ExampleObjectClass.cpp

```
void Student::accept(Visitor* v) {  
    v->visitStudent(this);  
}  
void Teacher::accept(Visitor* v) {  
    v->visitTeacher(this);  
}  
void Equipment::accept(Visitor* v) {  
    v->visitEquipment(this);  
}  
void Class::accept(Visitor* v) {  
    v->visitClass(this);  
}
```



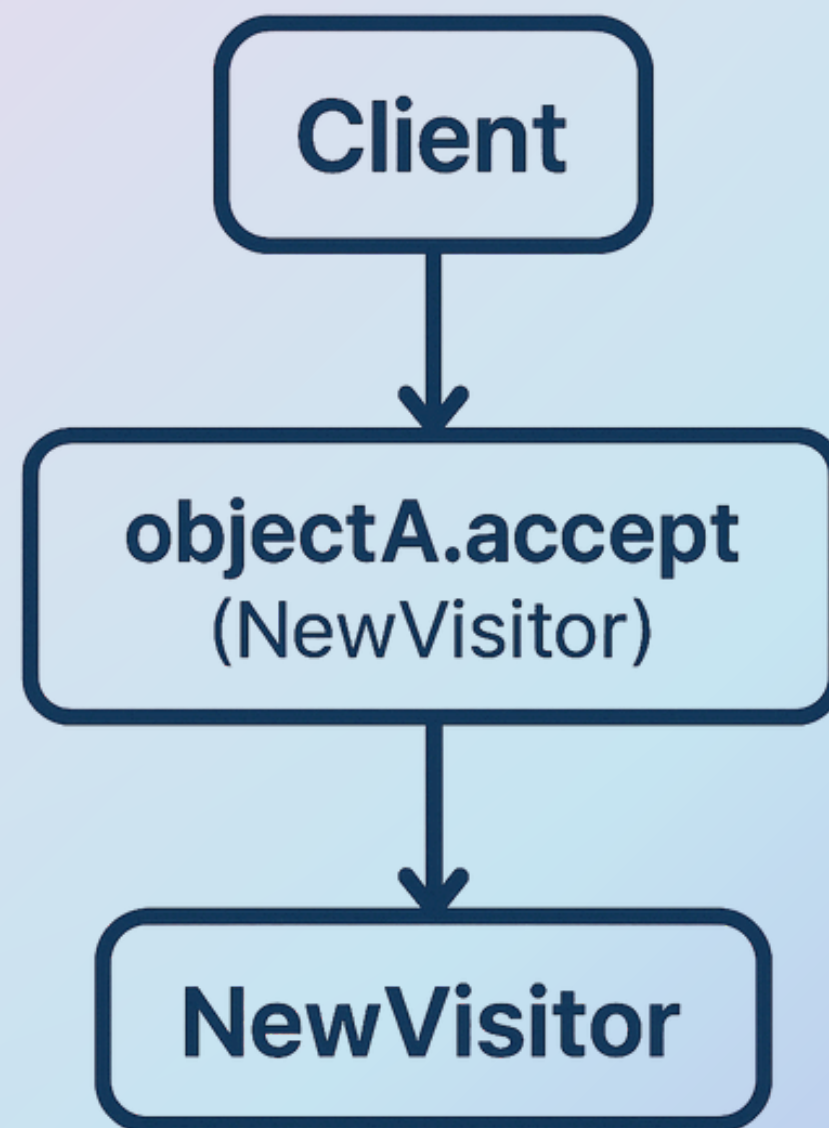
# Structure of the VISITOR Design Pattern

***Visitor Classes***

***Classes of objects  
in the object collection***

***The Client***

## *The Client*



- **The Client is the component responsible for organizing and coordinating** the use of the Visitor to perform operations on the Objects .
- The Client acts as the "user" of the design pattern, **ensuring that the Objects and the Visitor are properly connected** to execute the desired functionalities.

# The Client



## ExampleTheClient.cpp

```
class Application
{
private:
    std::vector<SchoolEntity*> entities;
public:
    ...
    void addEntity(SchoolEntity* entity);
    void exportToXML();
}
```

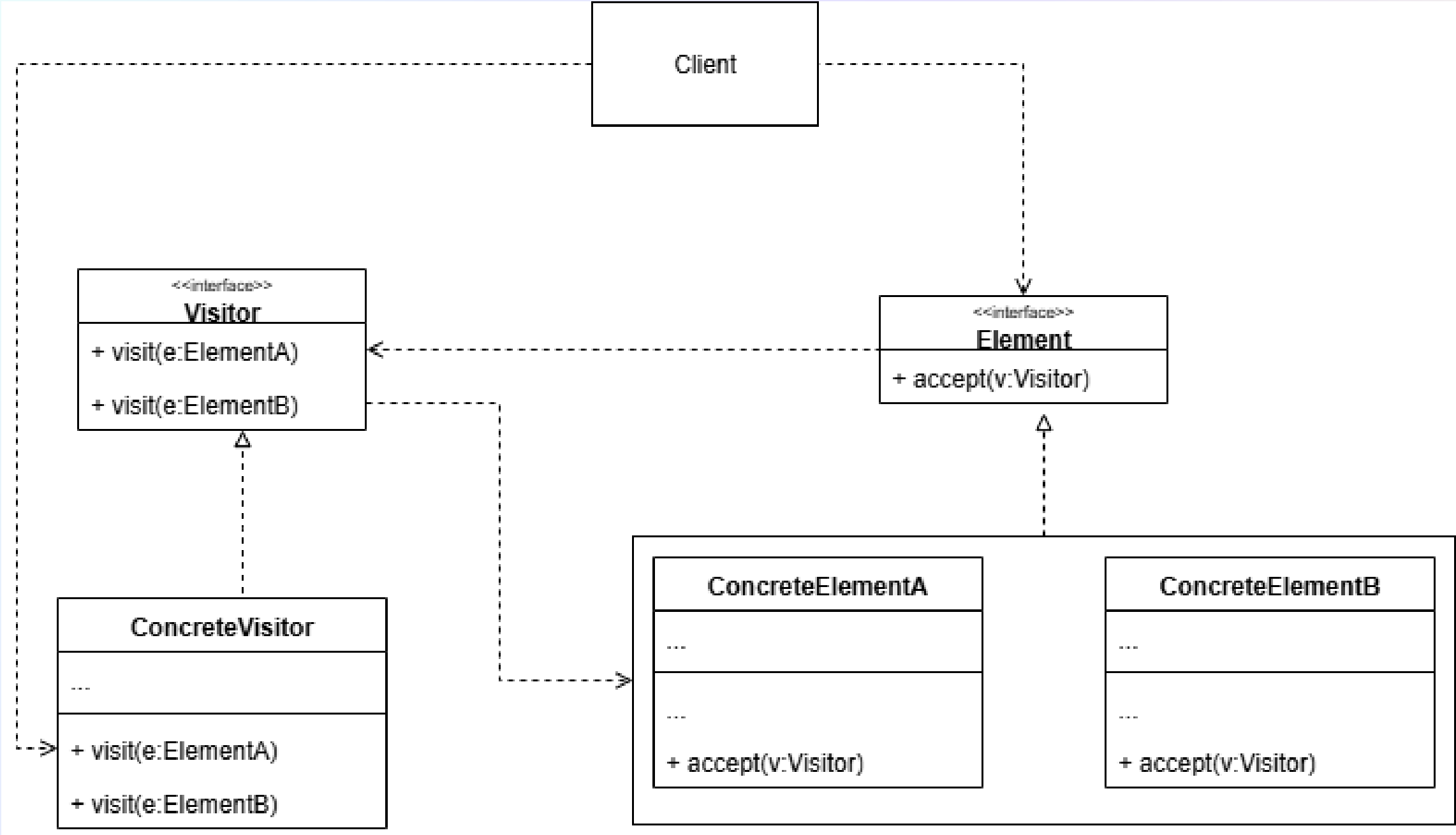
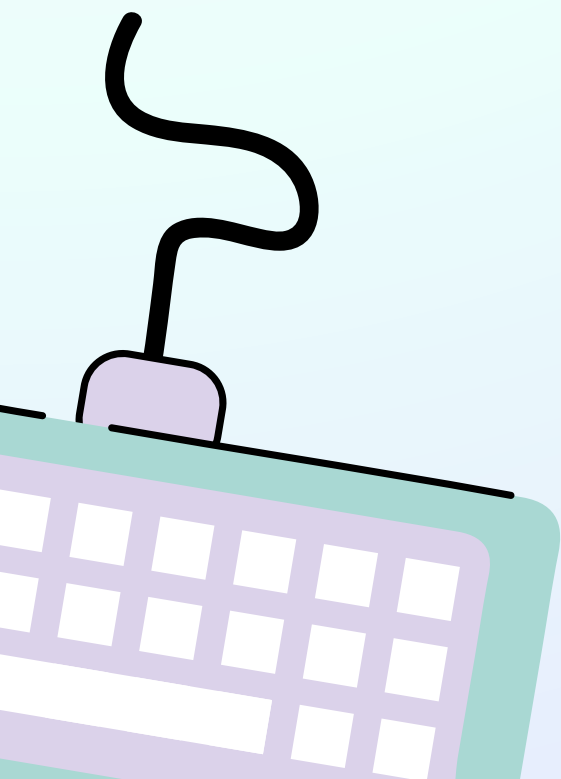


## ExampleMain.cpp

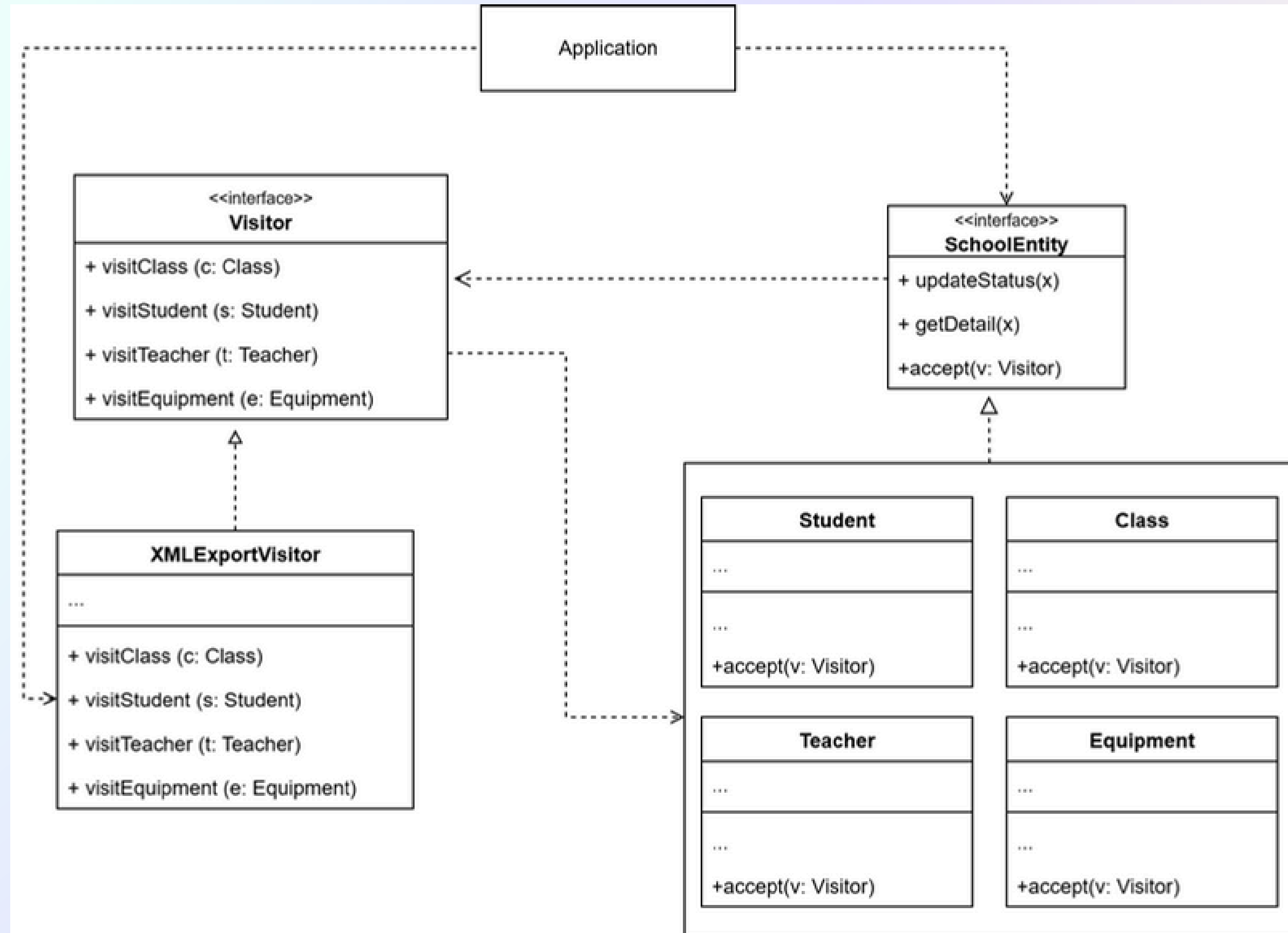
```
int main() {
    Application schoolApp;
    Class* scienceClass = new Class("Science 101");
    Student* student1 = new Student("S1001", "Alice");
    Teacher* teacher1 = new Teacher("T2001", "Physics");
    Equipment* microscope = new Equipment("E3001",
    "HPMicro");
    scienceClass->addMember(student1);
    scienceClass->addMember(teacher1);
    scienceClass->addMember(microscope);
    schoolApp.addEntity(scienceClass);
    schoolApp.exportToXML();
    ....
}
```



# A General Class Diagram

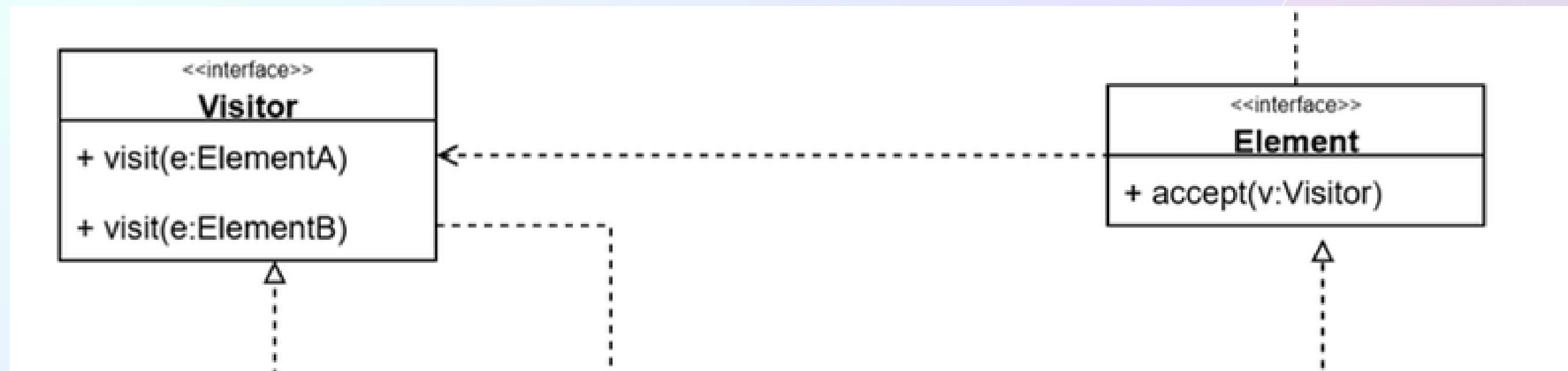


# Diagram for the problem



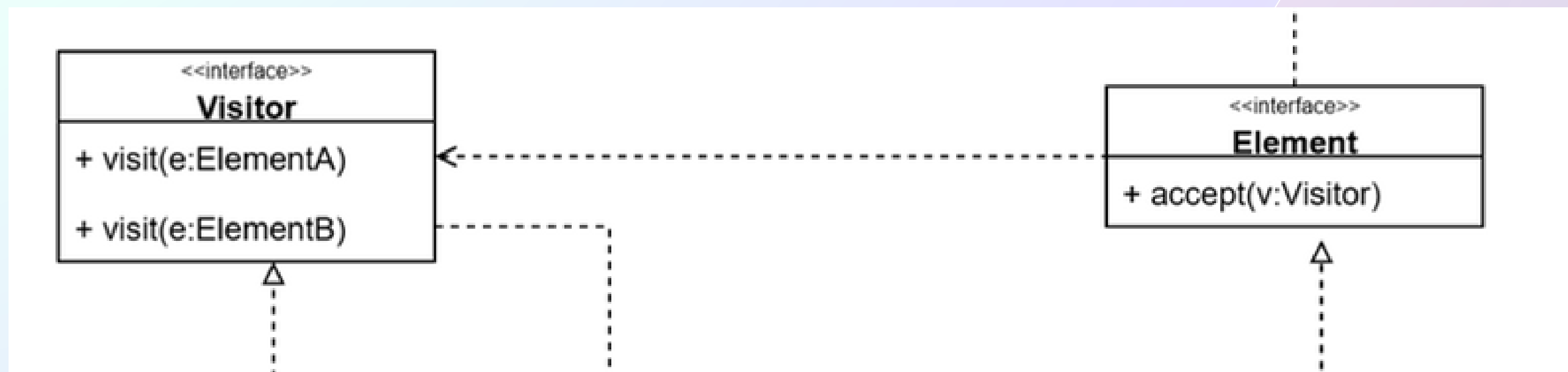
# Implementation

- 1 **Declare the visitor interface** with a set of “visiting” methods, one per each concrete element class that exists in the program.



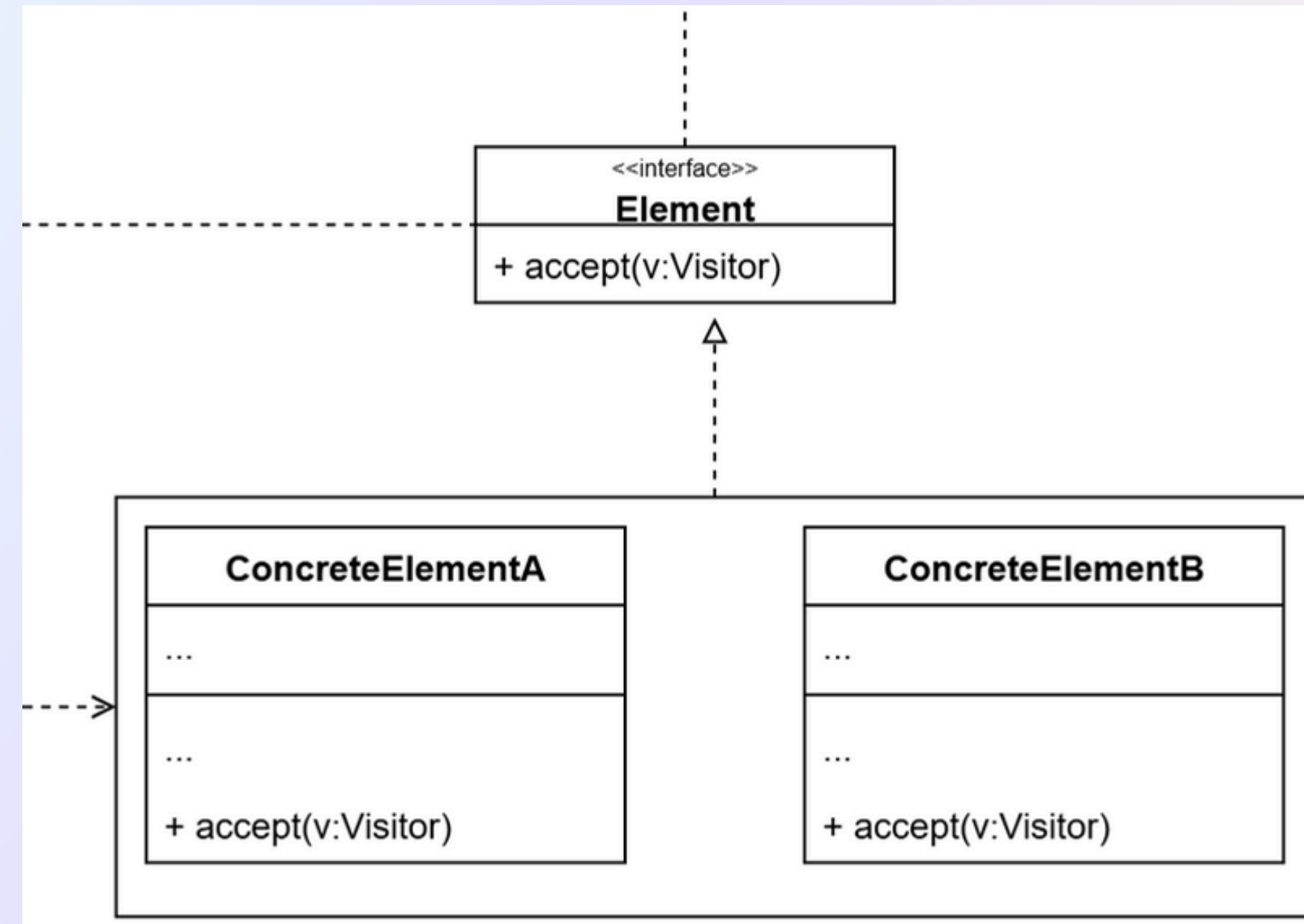
# Implementation

- 2 Declare the element interface.** If you're working with an existing element class hierarchy, add the abstract "accept" method to the base class of the hierarchy.



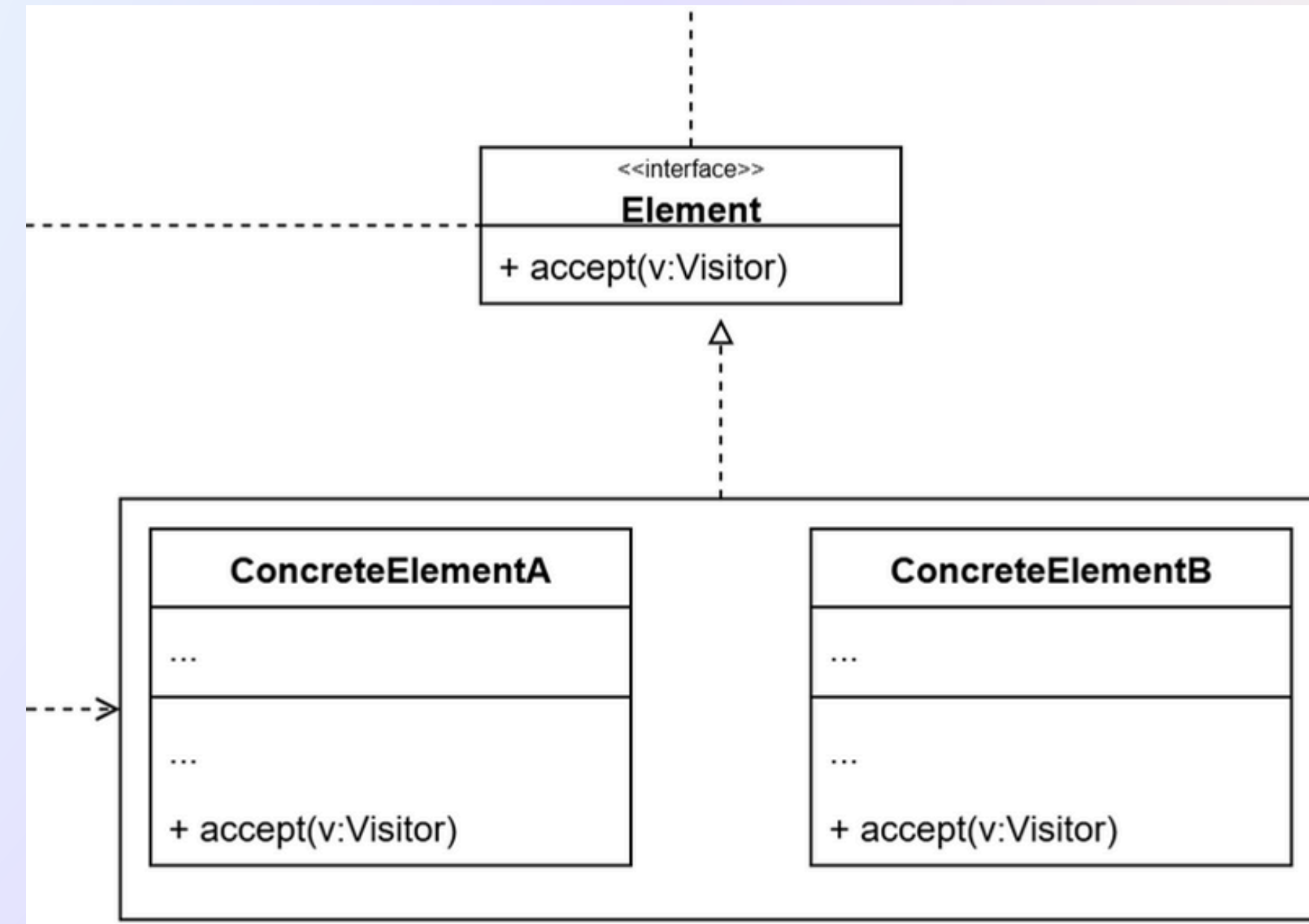
# Implementation

**3 Implement the accept methods in all concrete element classes.** These methods must simply redirect the call to a visiting method on the incoming visitor object which matches the class of the current element.



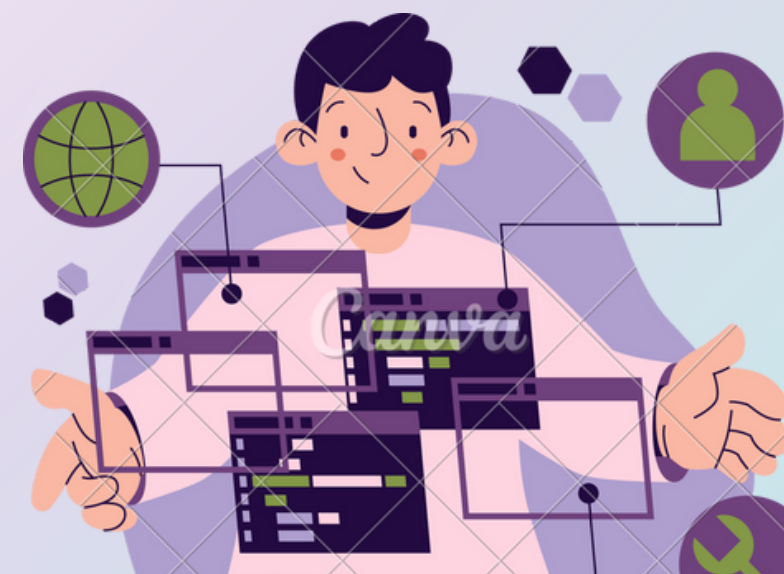
# Implementation

- 4 The element classes should only work with visitors via the visitor interface.** Visitors, however, must be aware of all concrete element classes, referenced as parameter types of the visiting method.



# Implementation

- 5 For each behavior that **can't be implemented** inside the **element** hierarchy, **create a new concrete visitor class** and implement all of the visiting methods.
- 6 The client must **create visitor objects** and **pass** them into elements via **"accept" methods**.



PROS

CONS



# PROS

1

**Open/Closed Principle:** You can introduce a new behavior that can work with objects of different classes without changing these classes.

2

**Single Responsibility Principle:** You can move multiple versions of the same behavior into the same class.

3

A visitor object can **accumulate some useful information** while working with various objects => handy when you want to traverse some complex object structure.

PROS

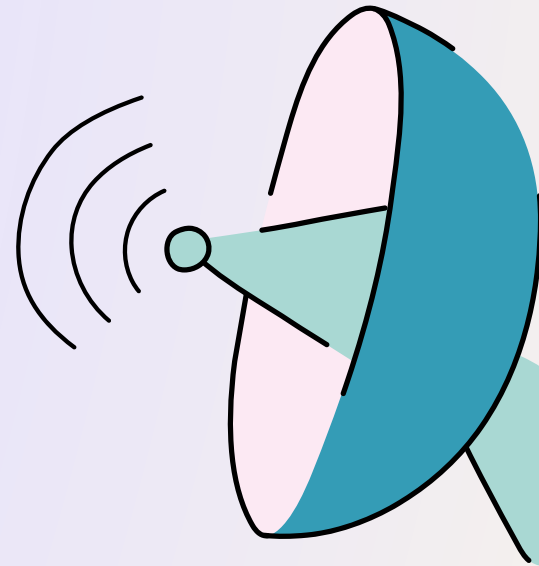
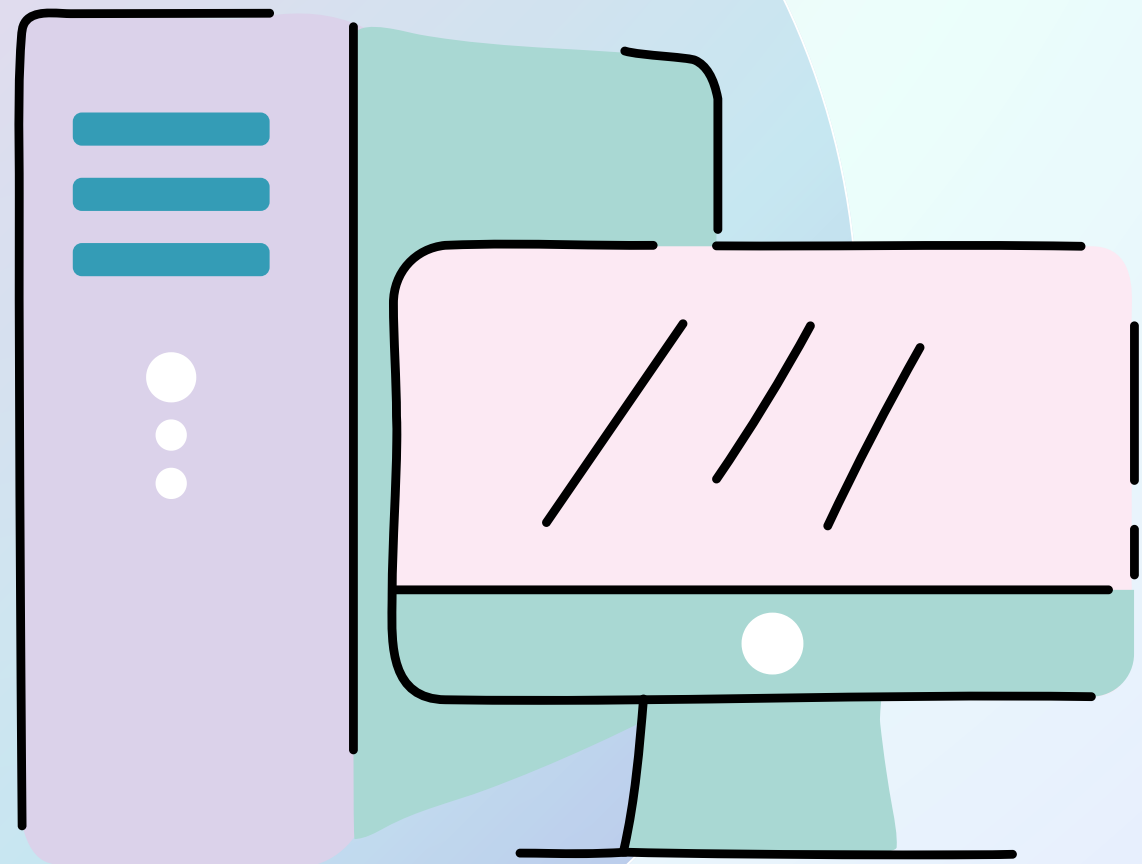
CONS



You need to **update all visitors** each time a class gets added to or removed from the element hierarchy.

Visitors might **lack the necessary** access to the private fields and methods of the elements that they're supposed to work with

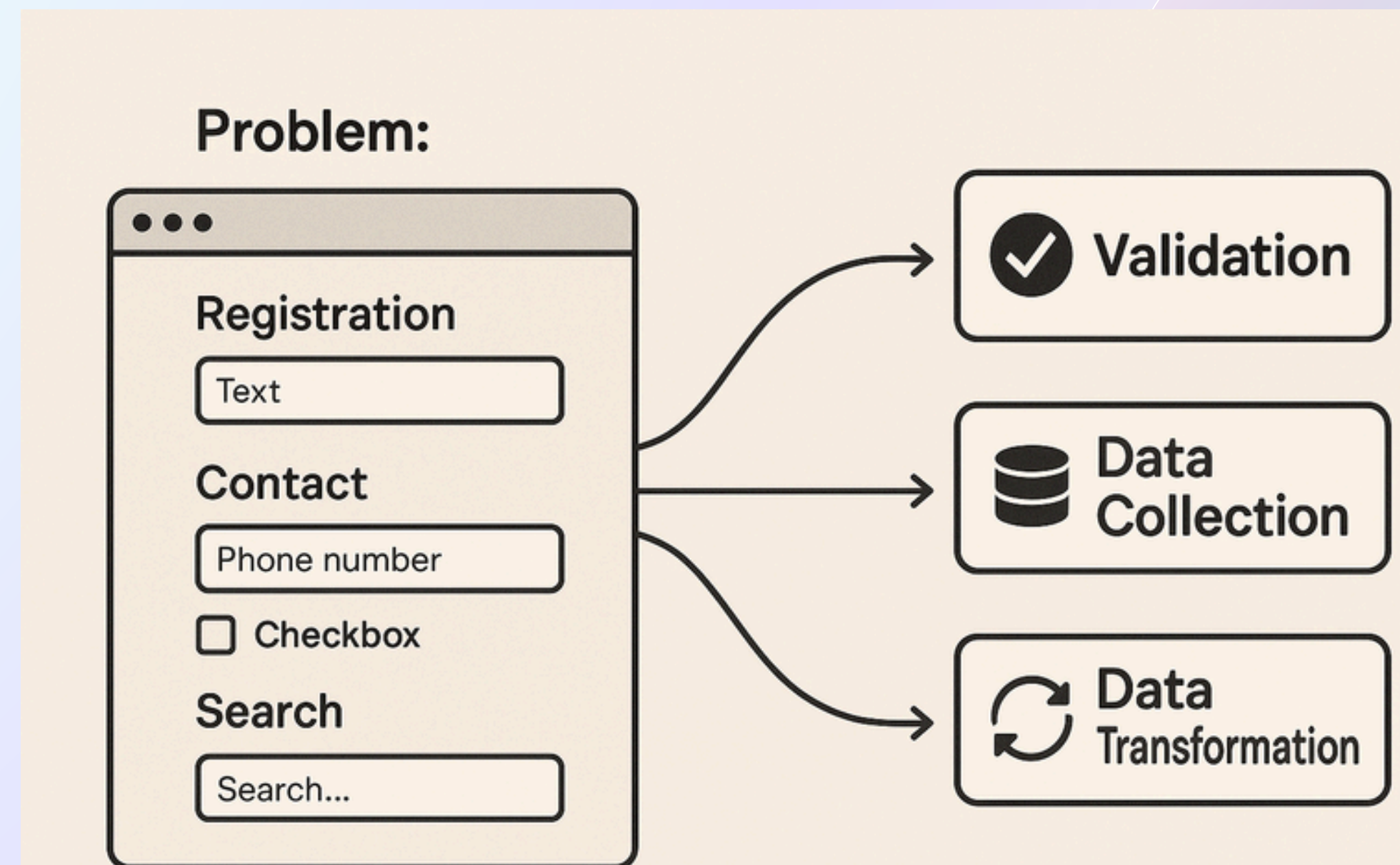
*CONS*



# Real-World Applications

# Web Development: Form Data Analysis and Processing

- **Problem:** You have various types of forms on a website . Each form contains different data fields. You need to perform different operations on the data submitted through these forms, such as:
  - **Validation:** Checking if the entered data meets specific criteria.
  - **Data Collection:** Extracting all the submitted values to send to the server.
  - **Data Transformation:** Converting data formats (e.g., date conversions).



# Web Development: Form Data Analysis and Processing

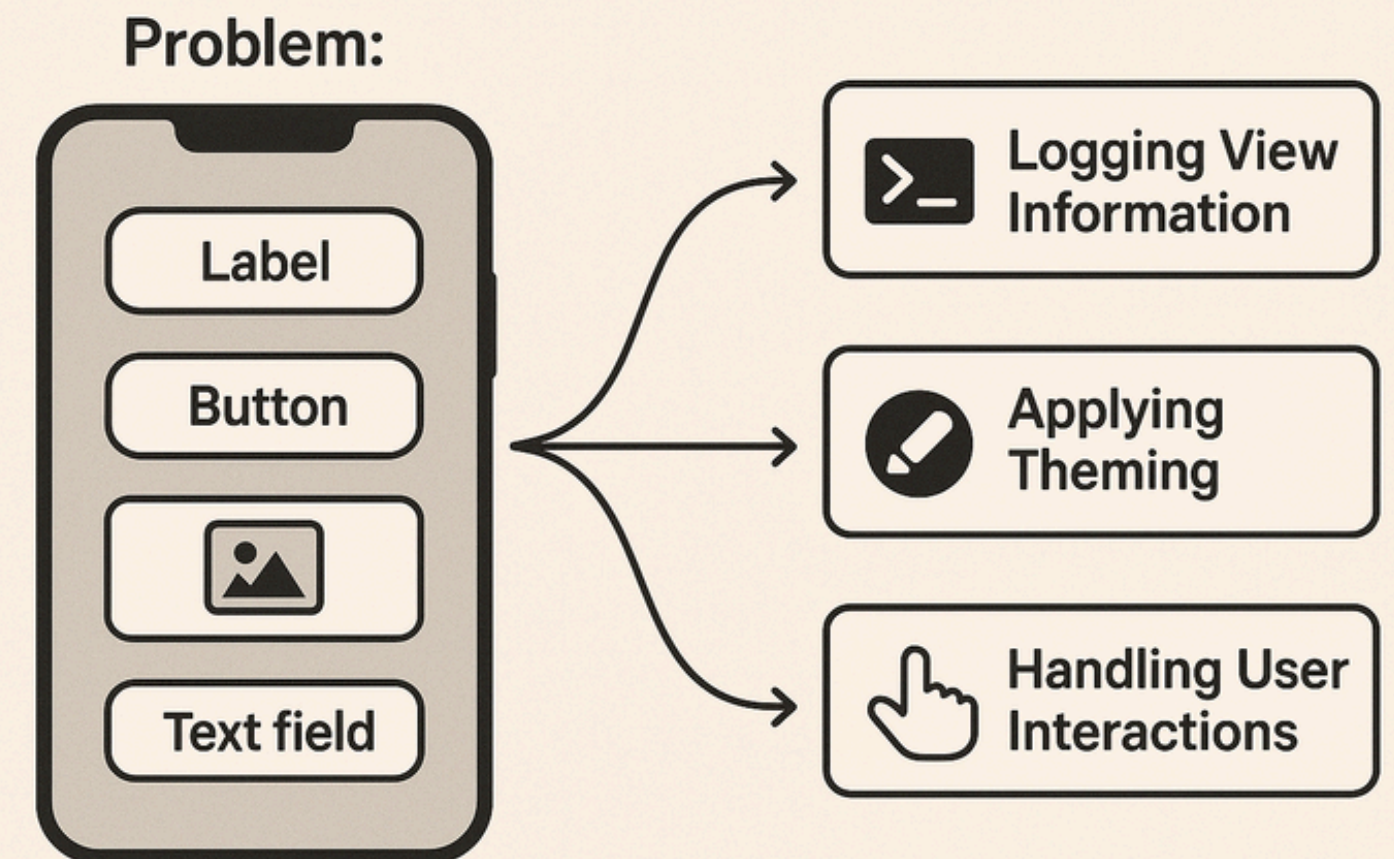
## How Visitor is Applied:

- **The form elements** (input text fields, select boxes, checkboxes, etc.) can be considered the **"Elements"**.
- **Each operation** (validation, collection, transformation) becomes a **"Visitor"**. Each visitor knows how to "visit" and perform its specific task on each type of form element.
- When you want to validate a form, you would **traverse the form's elements and have each element "accept" the validation visitor**. The visitor then applies the relevant validation rules for that type of field.

# Mobile Development: Rendering and Interacting with Diverse UI Elements

**Problem:** In mobile apps, you have a hierarchy of different UI elements. You might need to perform different actions on these elements, such as:

- **Logging View Information:** Printing details about each view (type, content, position).
- **Change theme:** Consistently changing colors, fonts, and sizes across relevant UI elements.
- **Handling User Interactions:** Setting up actions that occur when a user interacts with an element (e.g., tapping a button).

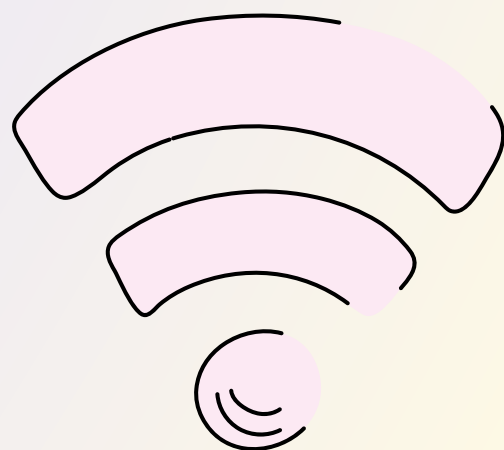
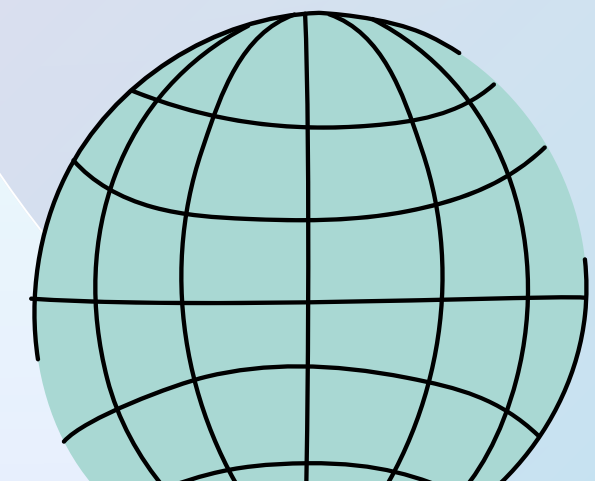


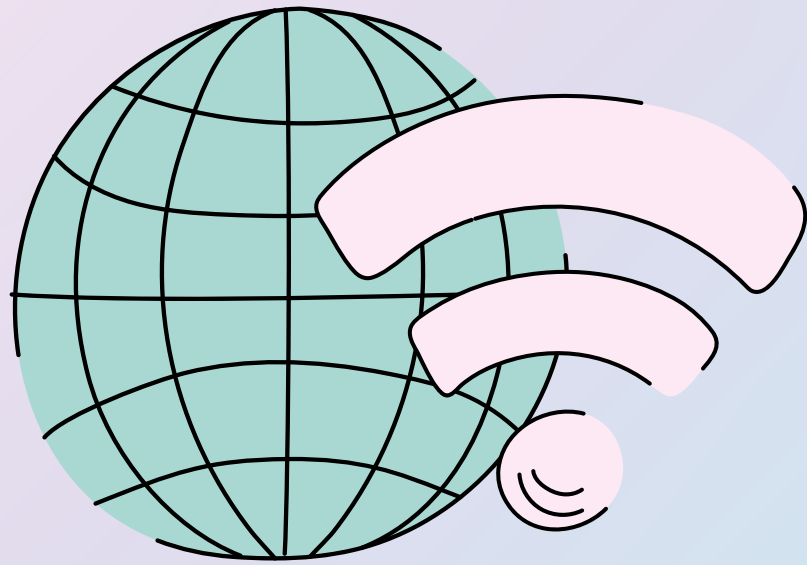
# Mobile Development: Rendering and Interacting with Diverse UI Elements

- **How Visitor is Applied:**

- **The UI views** (buttons, labels, image views, text fields, etc.) are the **"Elements"**.
- **Each action** (logging, theming, event handling setup) is a **"Visitor"**. Each visitor defines how it "visits" and acts upon each type of view.
- To log the entire UI hierarchy, you would **traverse the view tree** and each view would **"accept" the logging visitor**. This visitor would then print the appropriate information based on the view's type.

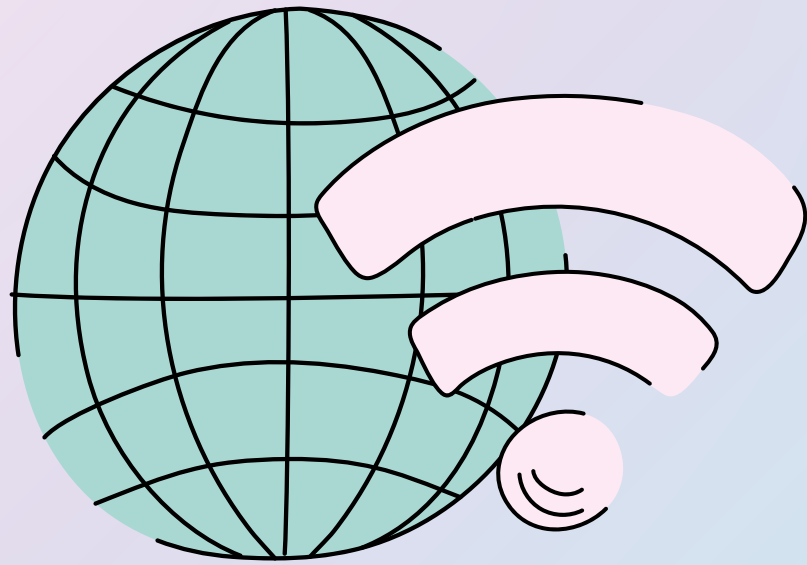
# Quiz





# When is the Visitor Pattern typically used?

- a) When you need to create objects with complex structures
- b) When you want to add new operations without modifying the object classes
- c) When you need to manage system resources
- d) When you want to hide the implementation details of a class



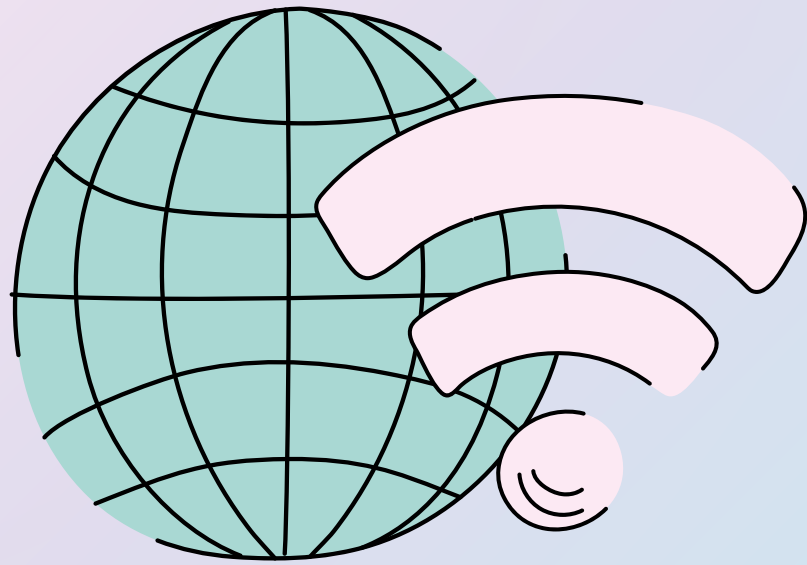
# When is the Visitor Pattern typically used?

a) When you need to create objects with complex structures

**b) When you want to add new operations without modifying the object classes**

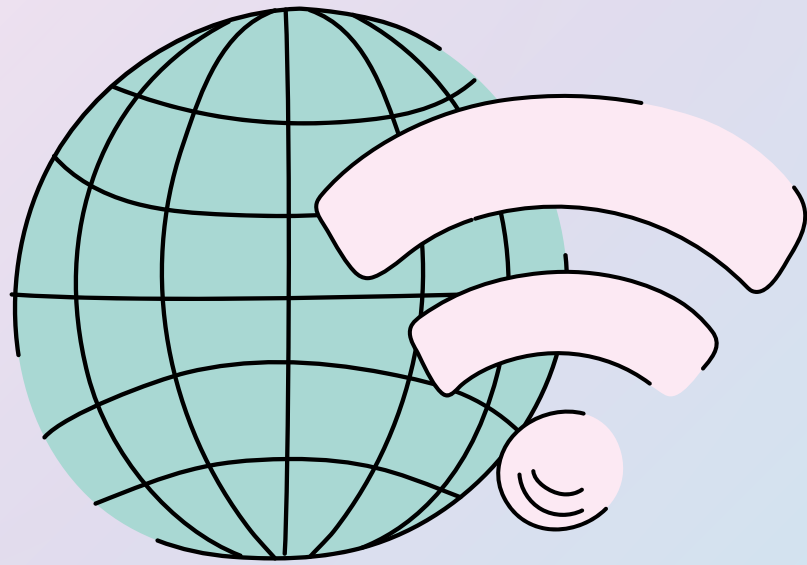
c) When you need to manage system resources

d) When you want to hide the implementation details of a class



# What is the main purpose of the Visitor Pattern?

- a) To create new objects flexibly
- b) To separate algorithms from the object structure
- c) To combine classes with similar structures
- d) To manage the lifecycle of an object



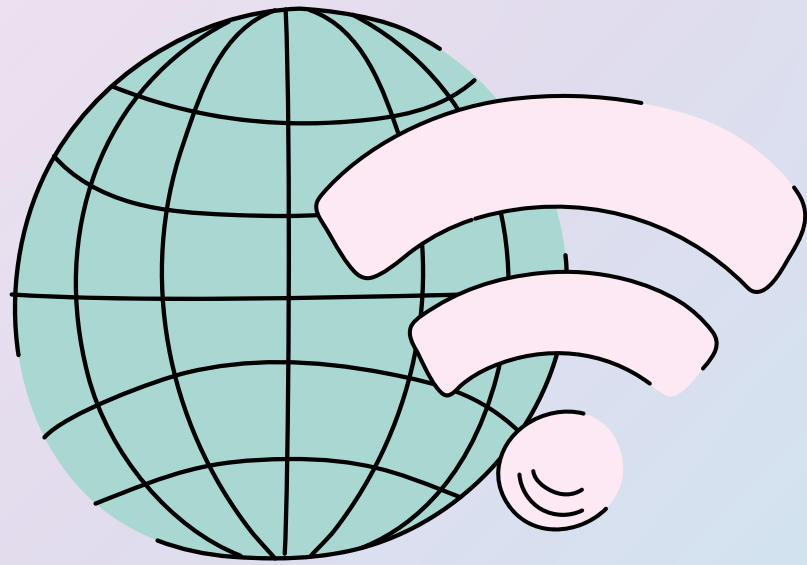
# What is the main purpose of the Visitor Pattern?

a) To create new objects flexibly

**b) To separate algorithms from the object structure**

c) To combine classes with similar structures

d) To manage the lifecycle of an object



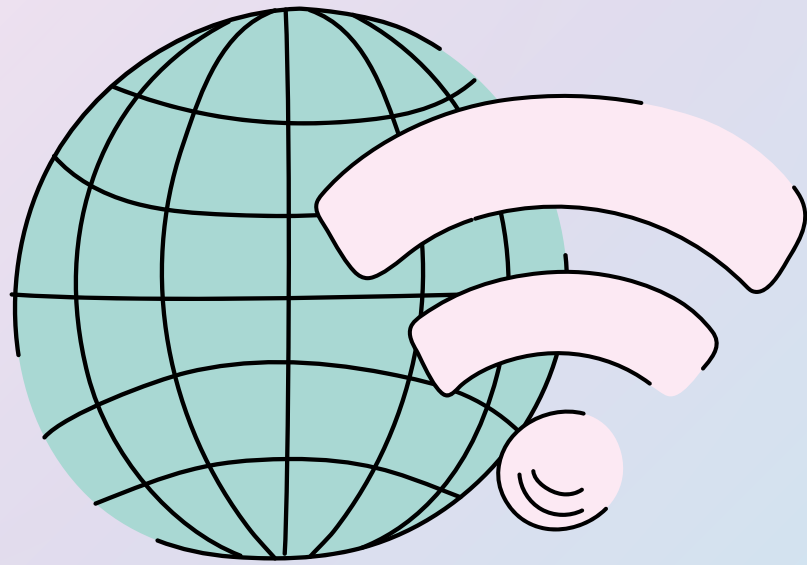
Which method in the Visitor Pattern is typically called by Element classes to accept a Visitor?

a) visit(visitor)

b) accept(visitor)

c) execute(visitor)

d) perform(visitor)



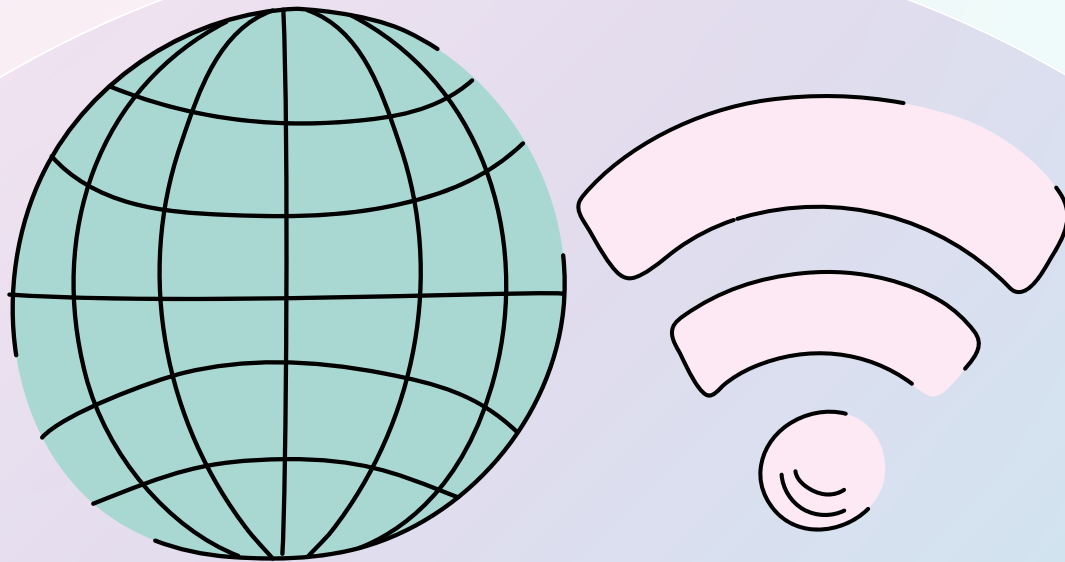
Which method in the Visitor Pattern is typically called by Element classes to accept a Visitor?

a) visit(visitor)

**b) accept(visitor)**

c) execute(visitor)

d) perform(visitor)



In the Visitor Pattern, which component is responsible for defining the interface for operations performed on elements?

a) Element

Carly

b) Visitor

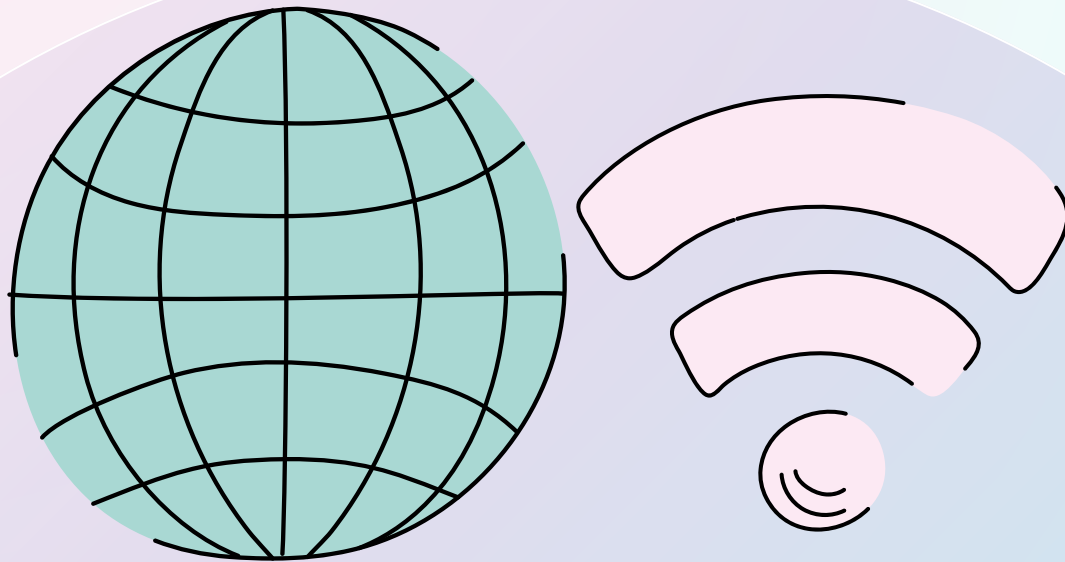
Carly

c) ConcreteElement

Carly

d) ObjectStructure

Carly



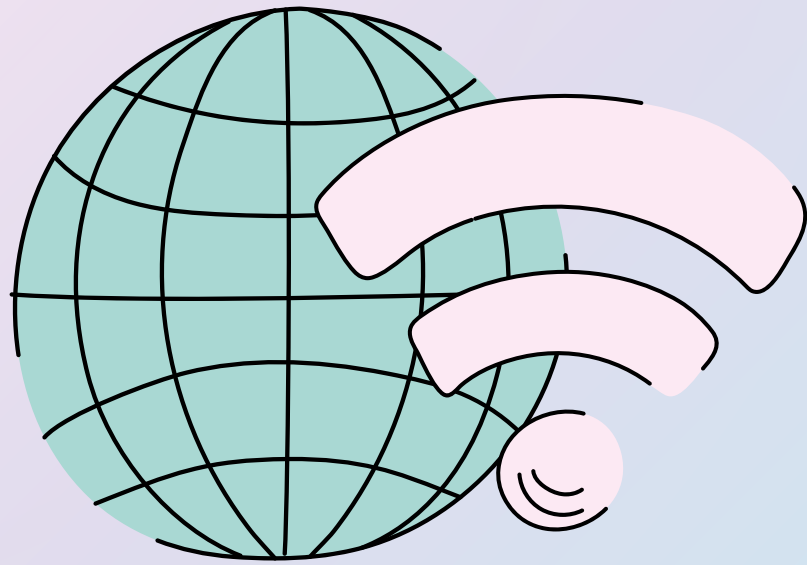
In the Visitor Pattern, which component is responsible for defining the interface for operations performed on elements?

a) Element

**b) Visitor**

c) ConcreteElement

d) ObjectStructure



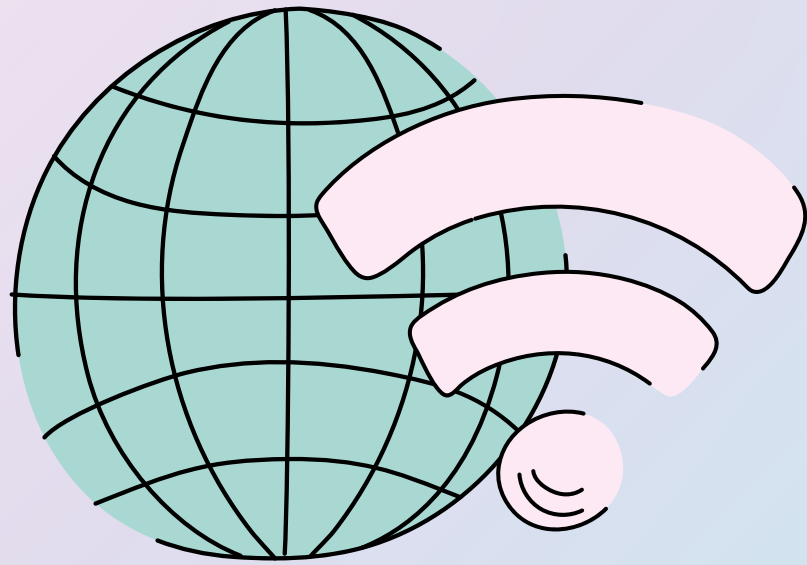
In C++, which OOP feature is commonly used to implement the Visitor Pattern?

a) Inheritance and Polymorphism

b) Encapsulation

c) Abstraction

d) Composition



In C++, which OOP feature is commonly used to implement the Visitor Pattern?

a) Inheritance and Polymorphism

b) Encapsulation

c) Abstraction

d) Composition

# Thank You

